

Evaluating Cloud Platform Performance for CNN and GAN Training: A Comparative Study of AWS, Google Colab, and Azure

Ahmed Iqbal, Musa Jawad, Alexander Sembrat, Touhidul Tonmoy

Preamble

Linked in this preamble is the github repo that contains the machine learning models used as well as other code used to analyze our results and create the graphs shown in this report.

<https://github.com/ahmedi64/SENG533>

Abstract— With the rapid adoption of machine learning (ML), cloud platforms have become essential for training, validating, and deploying deep learning models. This paper presents a comparative analysis of three major cloud platforms—Google Colab, Amazon Web Services (AWS), and Microsoft Azure—evaluating their performance when running Convolutional Neural Networks (CNNs) and Generative Adversarial Networks (GANs). Key performance metrics including training time, inference time, accuracy, and scalability are measured to determine the most effective platform for each model type. By using a consistent methodology across all platforms and models, and repeating experiments to ensure reliability, this study aims to provide actionable insights and recommendations for practitioners seeking optimal cloud solutions for deep learning tasks. While cost-efficiency was initially considered, it was excluded due to negligible variance at the test scale. The results offer a data-driven guide for selecting the best cloud platform based on model type and performance needs.

I. INTRODUCTION

Machine learning models are becoming ever more prevalent in the modern world and many organizations and individuals are leveraging them to great extent. Many of these corporations and individuals do not own their own hardware but instead use cloud services to train, validate, and deploy these models. With many different cloud service options being available, it is crucial to make an informed decision when deploying your model. That is why this paper will evaluate the performance of various deep learning algorithms on various cloud platforms.

The importance of this problem is clear as individuals and organizations would like to deploy their models on the fastest, most accurate, and most cost effective cloud platform available. This will

allow them to gain the greatest value from their model.

There have been a couple different evaluations of cloud platforms regarding machine learning in the past that explore the behaviour of CNN models given different CPU and GPU configuration as well as research into ease of use and performance using examples of what large organizations currently use cloud platforms for but they fail to provide any concrete data or make any true recommendation to the reader.

To fill this gap, we plan to evaluate both CNN and GAN models and compare their performance directly between three different cloud platforms. We will also make a recommendation based on our findings if one cloud platform is clearly the best in a certain aspect.

Some key questions that will be answered include: Which cloud platform is best for CNN's and GAN's? Which is fastest? Which is most accurate? Does training data size matter? To answer these questions, we will be using measurement techniques for our analysis.

The approach that will be followed to conduct this analysis is as follows. Start by selecting the cloud service platforms that will be tested. These will be determined by popularity and ease of access. Then select the deep learning models that will be tested on the services, these will be selected by a combination of popularity and complexity. Then for each cloud platform and model, the model will be trained, validated, then tested to judge its accuracy. Its training time will be recorded to compare to other cloud platforms. The CPU's and GPU's used

will also be recorded. Once the data has been recorded we will be able to compare the cloud services to each other and use statistical analysis to determine the best-performing service.

II. BACKGROUND AND RELATED WORK

The cloud services chosen for this test are as follows:

Google Colab. Google Colab was chosen because of its ease of use and free access to anyone. It is more simplistic than other cloud services and lacks fancy features to deploy the model long term but is good for an individual or an organization to use for testing.

Amazon Web Services (AWS). AWS was chosen because of its massive popularity with both organizations and individuals alike. AWS also provides lots of customizability to the user and allows for models to be deployed in various ways and to use varying levels of system resources. This service is a paid service.

Microsoft Azure. Azure was also chosen because of its popularity and features. Similar to AWS it has many features that allow the model to be trained and deployed, and it is also not a free service.

The deep learning models that will be used are Convolutional Neural Networks (CNN) and Generative Adversarial Networks (GAN).

CNNs are a type of neural network specifically designed for processing grid-like data, such as images. They use convolutional layers to extract features from the input data, reducing the number of parameters and enabling efficient computation. The CNN used for this experiment will be training on the MNIST number dataset [1][2].

GANs are a class of deep learning models that consist of two neural networks: a generator and a discriminator. The generator tries to generate realistic data samples, while the discriminator tries to distinguish between the generated samples and

real data. The GAN used for this experiment will be training on the MNIST fashion dataset [3][4].

The first piece of relevant literature compared various CPUs and GPUs performance on a cloud network that ran CNN's [5]. This paper did a good comparative analysis with CPUs and GPUs but did not specify any cloud platforms in general, we hope to do this. The second paper discussed the differences between various cloud platforms and their ML offerings [6]. We hope to expand to specific deep learning model performance which this paper did not do.

III. METHODOLOGY

Our methodology is simple as it relies on using measurements followed by statistical analysis to come to our conclusions.

System:

The system used can be defined as a deep learning model that is trained, validated, and tested on a cloud platform. While these actions are being processed, the key metrics will be recorded.

Key Metrics:

- Training Time - Time required to train a model (seconds/epoch)
- Inference Time - Time per sample during inference (ms/sample)
- Accuracy/SSIM Score - Final validation/test accuracy (%)

One thing to note is that originally, cost efficiency was planned as a key metric that would be measured, but after our testing, we realized that the cost was negligible for the amount of processing we were doing, so we could not take proper measurements to complete an analysis.

Important Factors:

- Cloud Platform - AWS, Google Colab, or Azure
- Computing Power - CPU/GPU/RAM allocated per task.
- Machine Learning Algorithm - CNN or GAN

Each cloud platform will have machine learning model code loaded onto it, then the trained data will be downloaded and input into the cloud service. Once the training data has been received, the model will be trained, validated, and tested. The accuracy and other metrics will all be measured and recorded for each model and cloud platform, respectively. Each test will be repeated 3 times and averaged for greater certainty in the results. Then, a statistical analysis comparing the cloud platforms will be conducted.

IV. RESULTS

Below are the results for each machine learning model and cloud service as previously outlined. At the end of the results, we will compare the platforms.

A. CNN

Google Colab Results:

We performed our procedure on Google Colab. Google Colab is free and has limited choices so it was as simple as loading the notebook and importing the libraries needed.. The model is then run. The results of our CNN model can be seen below:

Google Collab	All data	50% data	25% data	CPUs	RAM
Step Time	80.1 ms	83.3 ms	108.4 ms	2 cores	12.7 GB
Epoch Time	37.54 s	19.53 s	12.70 s	2 cores	12.7 GB
Total Time	1126.31 s	585.76 s	380.93 s	2 cores	12.7 GB
Test Loss	7.22	8.63	10.57	2 cores	12.7 GB
Test Accuracy	98.78%	98.79 %	98.70 %	2 cores	12.7 GB

Google Colab Results Dataset 1

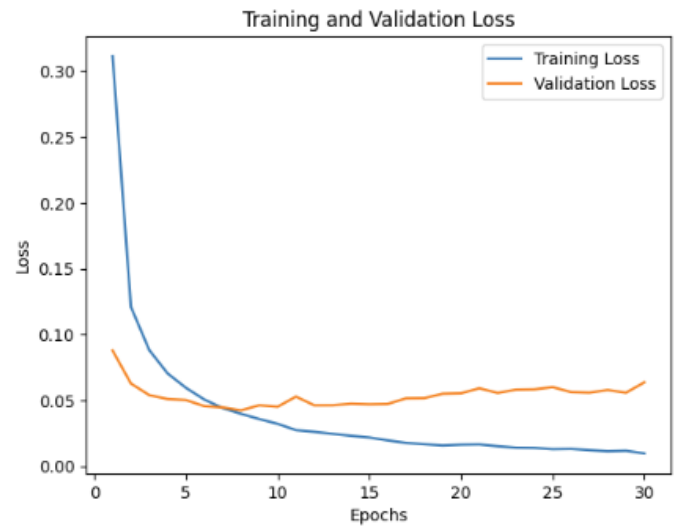


Fig. 1. Training and validation loss over 30 epochs for Google Colab

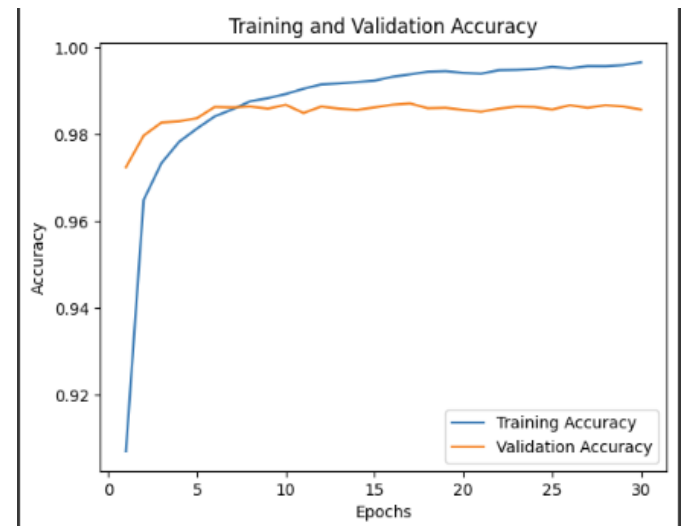


Fig. 2. Training and validation accuracy over 30 epochs for Google Colab

AWS Results:

For AWS we performed the same steps as before but using AWS Sagemaker, Amazon's AI and ML platform built into AWS. We used a ml.t3.medium instance which is the default provided by AWS and their lowest cost option. We then selected the pretuned "conda_tensorflow2_p310" kernel, which was set up to run our machine learning model. The model was loaded into the notebook and run. The results of our CNN model can be seen below:

AWS	All data	50% data	25% data	CPUs	RAM
Step Time	61.29 ms	65.47 ms	66.07 ms	2 cores	4 GB
Epoch Time	33.87 s	18.60 s	8.97 s	2 cores	4 GB
Total Time	1050 s	558 s	269 s	2 cores	4 GB
Test Loss	6.539	11.434	14.251	2 cores	4 GB
Test Accuracy	98.6	97.839	96.679	2 cores	4 GB

AWS Results Dataset 1



Fig. 3. Training and validation loss over 30 epochs for AWS

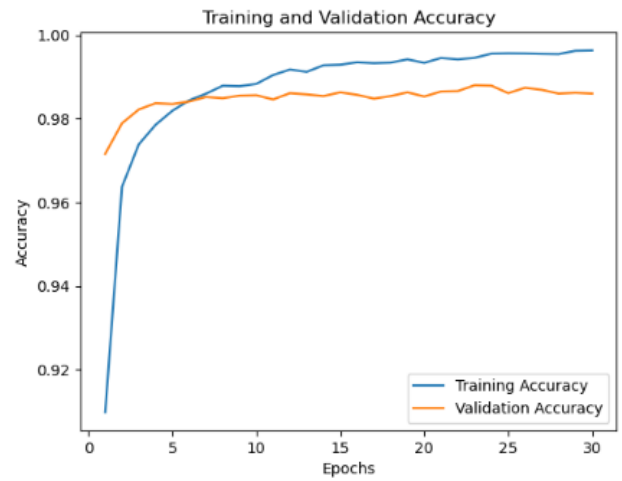


Fig. 4. Training and validation accuracy over 30 epochs for AWS

Azure Results:

We then performed the same steps and model on Azure AI using its “Standard_DS11_v2” computer. We uploaded the jupyter file along with some adjustments to Azure AI Notebook and selected the pretuned “Python 3.8 - Pytorch and Tensorflow” environment. This environment had both Pytorch and TensorFlow already tuned for jupyter use. The results of our CNN model can be seen below:

Azure	All data	50% data	25% data	CPUs	RAM
Step Time	36.5 ms	35.3 ms	38.9 ms	2 cores	14 GB
Epoch Time	17.13 s	8.27 s	4.5 s	2 cores	14 GB
Total Time	513.81 s	248.05s	136.7s	2 cores	14 GB
Test Loss	7.92	8.93	10.7	2 cores	14 GB
Test Accuracy	98.6	98.76	98.6	2 cores	14 GB

Azure Results Dataset 1

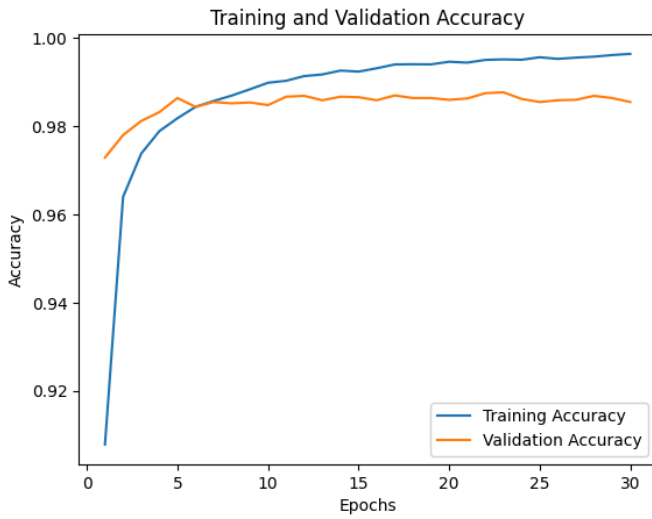


Fig. 5. Training and validation accuracy over 30 epochs for Azure AI

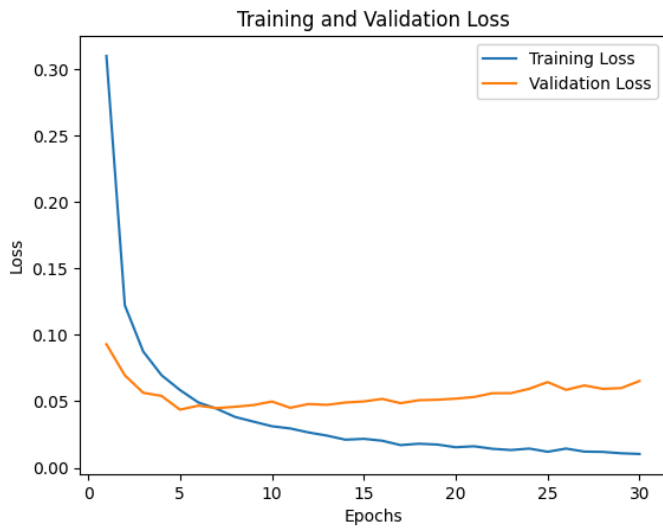


Fig. 6. Training and validation loss over 30 epochs

Final Comparison:

	AWS All data	Google Colab All data	Azure All Data
Step Time	61.29 ms	80.1 ms	36.5 ms
Epoch Time	33.87 s	37.54 s	17.13 s
Total Time	1050 s	1126.31 s	513.81 s
Test Loss	6.539	7.22	7.92
Test Accuracy	98.6	98.78	98.6
CPU	2	2	2
RAM	4 GB	12.67 GB	14 GB

From our table we can see that for our CNN model, Azure AI was the quickest. It had a step time of 36.5ms which is less than half of Google Collab, and the total time also reflected this. We can also see that even though it was the quickest, Azure AI still managed to get accuracy scores of 98.6%. This shows us that Azure AI is not sacrificing speed for accuracy. Azure AI also had the same number of cores as Google Collabn and AWS, while having 14 GB of ram. However looking at the other results, we can see that even though Google Collab had the second most RAM, at 12.67 GB, it still placed last in terms of speed. Being beaten by AWS which only had 4GB of ram. This indicated to us that RAM was not a limiting factor, or a factor which makes a huge difference when training CNN models. This indicates to us that the CPU is the bottleneck for each case. Google collab CPUs are shared resources. This means that other users are also using these resources and unless you pay for Collab Pro +. This explains why Google Collab had the worst scores in terms of Total and Epoch times. The AWS CPU should theoretically be the best, as they use the most powerful CPU between our 3 competitors, the Intel Xeon Platinum 8175M [8].

However AWS has a burstable cpu [7]. Essentially, the CPU runs on a baseline performance which is much lower than the full potential of the CPU. The CPU can “burst” above this baseline when you have “CPU Credits”. As such when you don't have these CPU Credits, you are not fully utilizing the CPU. Azure on the other hand has a non-burstable CPU. This means we are always able to utilize 100% of the CPU. This can be see in the below image where the CPU utilization is at 100% and the RAM utilization is at 11% when doing our Epoch steps:

seng2core · Kernel busy CPU100% RAM 11%

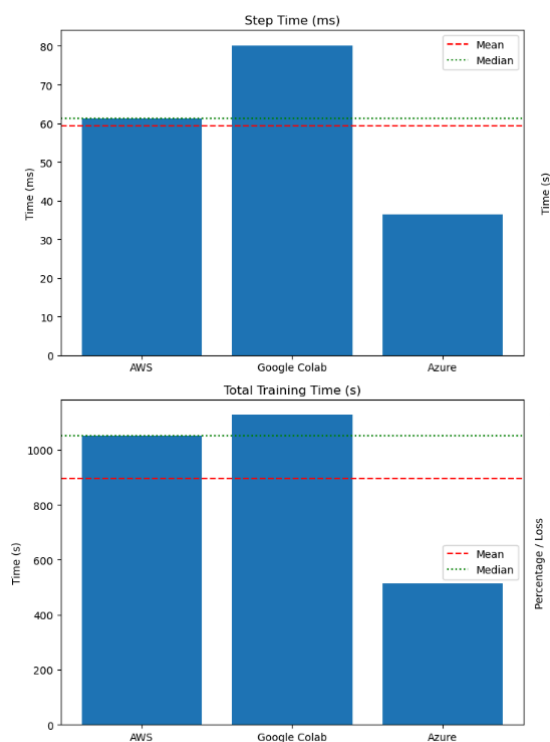


Fig. 7. Summary of Step and Training Times.

	Mean	Median	Variance
Step Time	59.3 ms	61.29 ms	478.22 ms
Epoch Time	29.51 s	33.87 s	118.4 s
Total Time	896.71 s	1050 s	111413.2 s
Test Loss	7023	7022	0.48
Test Accuracy	98.66	98.60	0.01

CNN Statistics Summary

The tables and graphs above show a summary of the results and make it clear that Azure has by far the fastest baseline configuration available for CNN models.

B. GAN

Google Colab Results:

We performed our procedure on Google Colab. As mentioned before, Google Colab is free and has limited choices so it was as simple as loading the notebook and importing the libraries needed. The model was then run. The results of our GAN model can be seen below:

Google Colab	All data	CPUs	RAM
Step Time	276.89 ms	2 cores	12.7 GB
Epoch Time	650.70 s	2 cores	12.7 GB
Total Time	6507.3 s	2 cores	12.7 GB
SSIM Score	0.1510	2 cores	12.7 GB

Google Colab Results Dataset 2



Fig. 8. Generator training loss over 10 epochs

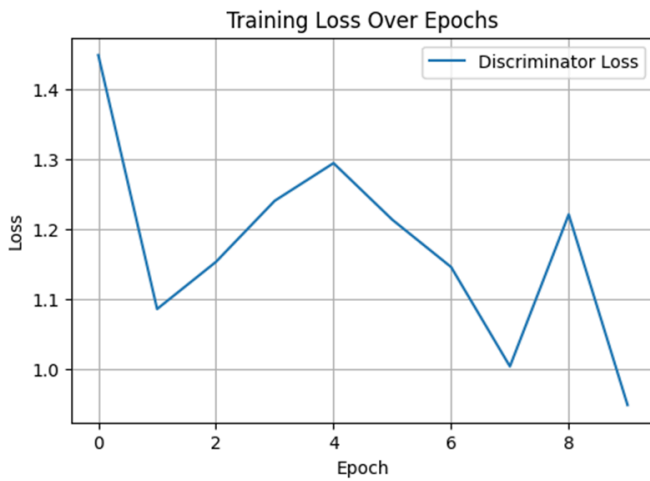


Fig. 9. Discriminator training loss over 10 epochs

AWS Results:

For AWS we performed the same steps as before with the CNN for AWS. We used a ml.t3.medium instance, which is the default provided by AWS and their lowest cost option. We then selected the pretuned “conda_tensorflow2_p310” kernel, which was set up to run our machine learning model. The model was loaded into the notebook and run. The results of our GAN model can be seen below:

AWS	All data	CPUs	RAM
-----	----------	------	-----

Step Time	255.31 ms	2 cores	4 GB
Epoch Time	601.21 s	2 cores	4 GB
Total Time	60012	2 cores	4 GB
SSIM Score	0.1778	2 cores	4 GB

AWS Results Dataset 2

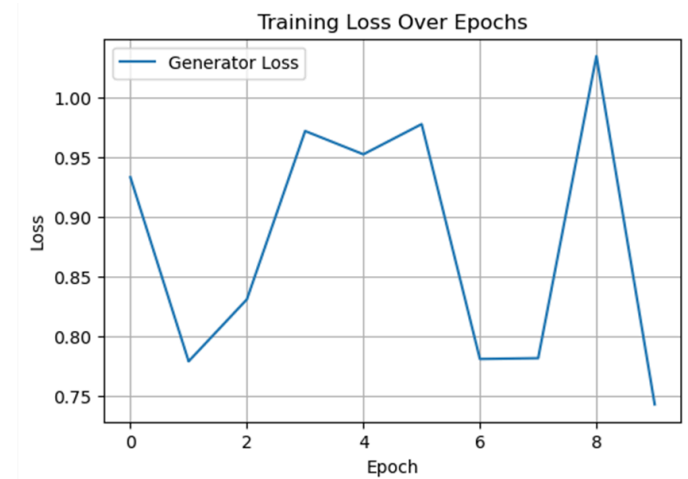


Fig. 10. Generator training loss over 10 epochs

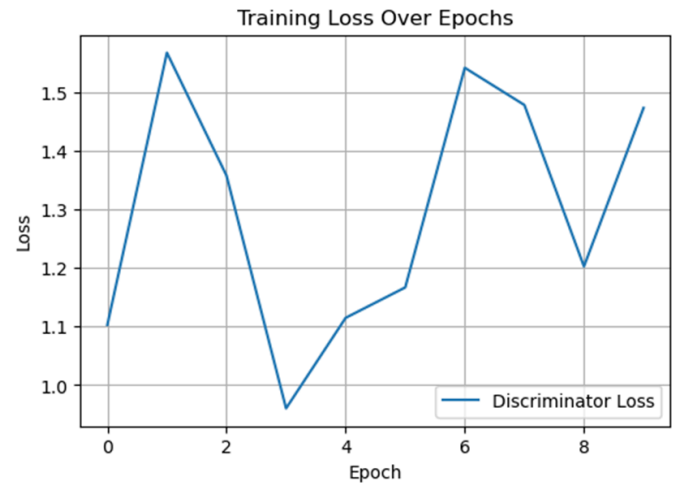


Fig. 11. Discriminator training loss over 10 epochs

Azure Results:

As with the CNN, we used the “Standard_DS11_v2” computer on Azure AI. We uploaded the jupyter file and selected the “Python 3.8 - Pytorch and Tensorflow” environment. The results of our CNN model can be seen below:

Azure	All data	CPUs	RAM
Step Time	90.17 ms	2 cores	14 GB
Epoch Time	211.90 s	2 cores	14 GB
Total Time	2118.96 s	2 cores	14 GB
SSIM Score	0.163	2 cores	14 GB

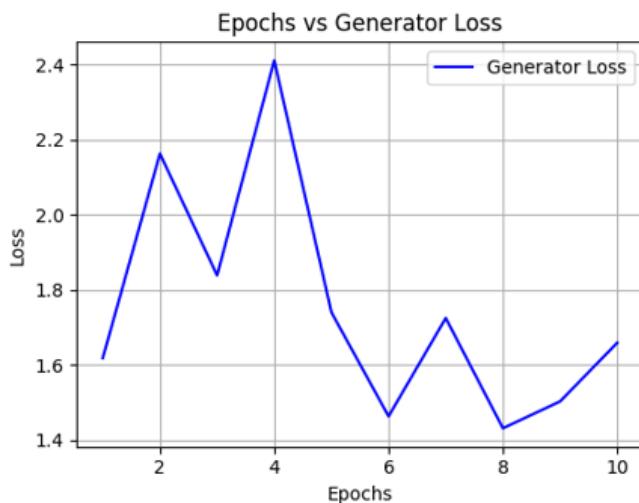


Fig. 12. Generator training loss over 10 epochs

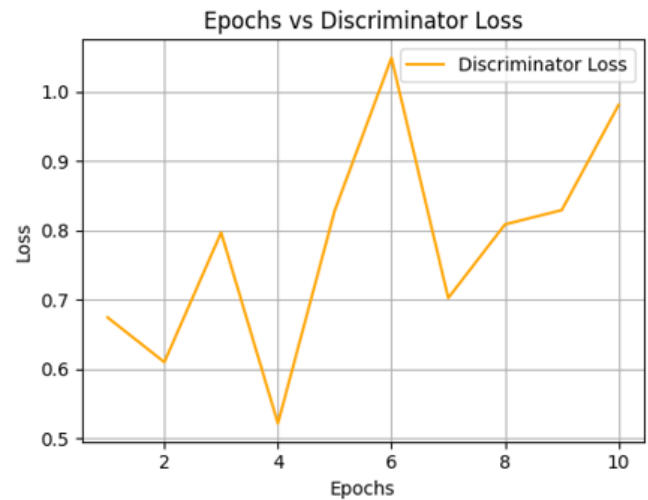


Fig. 13. Discriminator training loss over 10 epochs

Final Comparison:

	AWS All data	Google Colab All data	Azure All Data
Step Time	255.31 ms	276.89 ms	90.17 ms
Epoch Time	601.21 s	650.70 s	211.90 s
Total Time	60012	6507.3 s	2118.96 s
S SIM Score	0.1778	0.1510	0.163
CPU	2	2	2
RAM	4 GB	12.67 GB	14 GB

From the table above, we can observe that Azure once again outperformed both AWS and Google Collab across all speed-related metrics with our GAN model. Azure achieved the fastest step time at 90.17 m, which is significantly quicker than AWS (225.31 ms) and Google Collab (276.89 ms). This trend continues with epoch time and total training time, where Azure had about $\frac{1}{3}$ of AWS and Google Collabs times. Despite the differences in runtime, the SSIM score, which represents the actual image similarity was quite close. AWS slightly had an edge over the others with a score of

0.1778. Azure still maintained a good SSIM score of 0.163 with drastically faster training. Google Colab once again was the slowest, due to the shared CPU environment, despite having more RAM than AWS. This aligns with our earlier CNN findings and can be attributed to AWS using a burstable CPU model, which does not consistently operate at full performance without CPU credits. This suggests that for GAN model training, consistent CPU access has a greater impact on performance than the overall RAM capacity.

V. CONCLUSIONS AND FUTURE WORK

This study evaluated the performance of CNN and GAN models across three popular cloud platforms, Google Colab, AWS, and Azure, using metrics such as training time, inference time, accuracy, and scalability. Our findings offer a practical comparison for individuals and organizations seeking to choose the most effective platform for deploying deep learning models. While all platforms demonstrated strengths in different areas, our results provide a clear performance-based recommendation depending on specific model requirements and resource constraints.

Future work could expand this analysis by incorporating additional model types, such as transformers or reinforcement learning agents, and by exploring more intensive workloads to assess cost-efficiency at scale. Further, investigating automated deployment pipelines, long-term usage implications, and environmental impact could provide a more holistic view of cloud-based ML solutions. Integrating real-world datasets and examining transfer learning performance across platforms may also enhance the applicability of future findings. Additional future workloads include evaluating the higher tiered services that use more CPU/GPU/RAM, this was part of our original scope but our AWS setup had issues and used all \$25 of our credit when we attempted to use higher cores even though it was failing to actually run our models.

Overall, Azure seems to come out as the winner. It is by far the fastest of the 3 base services and the accuracy was very similar across all 3 services.

Azure also allows you to deploy your model in a variety of ways. Google Colab is also a decent pick if you need a quick and easy way to prototype your model, and this leaves AWS coming in last being the least easy to work with and not being free.

REFERENCES

- [1] H. Khodabakhsh, "Mnist dataset," Kaggle, <https://www.kaggle.com/datasets/hojjatk/mnist-data-set/data> (accessed Mar. 25, 2025).
- [2] Aruneshhh, "CNN model (accuracy >98) -mnist," Kaggle, <https://www.kaggle.com/code/aruneshhh/cnn-model-accuracy-98-mnist/notebook> (accessed Mar. 25, 2025).
- [3] M. H. Abir, "DCGAN | Deep Convolutional GAN," Kaggle, Apr. 2025. [Online]. Available: <https://www.kaggle.com/code/mdismielhossenabir/dcgan-deep-convolutional-gan>
- [4] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms," *Kaggle*, 2017. [Online]. Available: <https://www.kaggle.com/datasets/zalando-research/fashionmnist>
- [5] M. S. Rana, M. M. Rahman and M. M. Rahman, "A Comparative Analysis of CPU and GPU-Based Cloud Platforms for CNN Binary Classification," *ResearchGate*, Apr. 2024. [Online]. Available: https://www.researchgate.net/publication/382302761_A_Comparative_Analysis_of_CPU_and_GPU-Based_Cloud_Platforms_for_CNN_Binary_Classification
- [6] M. R. Islam and M. M. Rahman, "A Comparative Study of Cloud Platforms for Deep Learning: CPU vs GPU," *TechRxiv*, Apr. 2024. [Online]. Available: <https://www.techrxiv.org/doi/full/10.36227/techrxiv.173611042.27406103/v1>

[7]“Key concepts and definitions for burstable performance instances - Amazon Elastic Compute Cloud,”docs.aws.amazon.com.

Available:

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/burstable-credits-baseline-concepts.html>

[8]AWS, “Amazon EC2 T3 Instances – Amazon Web Services (AWS),” Amazon Web Services, Inc., 2019.

<https://aws.amazon.com/ec2/instance-types/t3/>